CARELON GLOBAL SOLUTIONS · BITS

RTM Console
Backend Implementation Guide

The changes we will make to give the console a database: PostgreSQL on AWS, migrations with Kysely run by a single **db:migrate** command, a validated configuration, and how the rules engine saves and recovers its work.

Real-Time Monitoring & Escalation

FOR	COVERS	BUILDS ON	EDITION
Backend engineers · BITS	Database, migrations, configuration, deployment	PostgreSQL Schema (12 Oct)	27 October 2026

PREPARED BY

Arielle Jimera

UI/UX Designer / Developer · BITS Automation Team

Contents

1	Before we start	2
2	What changes, at a glance	3
3	What gets saved, and when	5
4	Hosting on AWS	8
5	Configuration: one validated environment	14
6	Connecting to PostgreSQL	15
7	Migrations with Kysely	17
8	Running db:migrate before a deployment	19
9	Changes to the proposed schema	21
10	How the engine saves and recovers	22
11	Local development and tests	24
12	Delivery plan and acceptance checks	25
13	Frequently asked questions	25
14	Quick reference card	27

1 Before we start

Today the console saves nothing: rules, call-outs, strikes and incidents are lost every time it restarts. This guide adds a database. The first part of the work can start now. The questions that shape the tables are all answered below.

Can start now

- [ ] Check all settings when the app starts, and stop with a clear message if one is missing (Section 5).
- [ ] Set up the database and the `npm run db:migrate` command (Sections 4, 7 and 8).
- [ ] Save the rules, so rule changes survive a restart (Section 3).

Questions

Question	Details	Who answers	Needed by
1. What is a shift, and when do strikes start again at zero? Decision 7	Answered A shift is one day on US Eastern time, midnight to midnight. Strikes start again at zero at Eastern midnight. The floor works within one Eastern day, so no shift crosses midnight; the overlap only appeared when counted in Philippine time.	Floor lead	Phase 2
2. When is an incident opened?	Answered On every strike from 3 on. Each one opens its own incident to investigate, so an agent who reaches strike 5 on the same rule in a day has 3 incidents. Incidents never merge, and one left open from an earlier day is not touched.	Floor lead	Phase 2
3. How long do we keep raw Genesys events? Decisions 8 and 9	Answered One day. Raw updates from Genesys ("went on break", "took a call") are saved in `source_event_stream` so the day's KPIs survive a restart. At midnight Eastern the table is emptied and the KPIs start again at zero, like strikes. Call-outs and incidents are kept separately and are not affected.	Team lead	Phase 3

> **ALSO NEEDED: IDS INSTEAD OF NAMES**
>
> The console links agents and teams by name, so two people with the same name, or a renamed team, would mix up their records. Before people are saved (Phase 1), the console switches to Genesys IDs, which never change. This is coding work we already know about (backlog item 4), not a question.

Setup checklist

- [ ] **Confirm the app's server**: most likely the existing EC2 instance (Section 4).
- [ ] **Create the database and its two logins** on `van-reservation-prod`, once, in pgAdmin (Section 4). DONE 2026-11-03, steps 1 to 7; the test login as `rtm_app` (step 8) is still to do.
- [ ] **Note the PostgreSQL version** (step 1 of the pgAdmin steps), so local development uses the same one (Section 11). DONE PostgreSQL 17.7; the local Docker image is `postgres:17.7-alpine`.
- [ ] **Write the deploy script** that runs `db:migrate` before restarting the app (Section 8).

2 What changes, at a glance

Every change in this guide, in one table. New pieces are marked NEW, changes to existing code CHANGE, corrections to the proposed schema FIX, and pieces that wait on a decision WAITS.

Change	What it means
Typed environment NEW	One module, `lib/env.ts`, validates every setting with Zod and is the only place that reads `process.env`. A missing or invalid setting stops the server at start-up with one message listing every problem (Section 5).
Validation at boot NEW	`instrumentation.ts` parses the environment in its `register()` hook, which Next.js runs once before the server accepts requests. The Node-only part is in `instrumentation-node.ts`, so the edge build never sees it.
Database URL module NEW	`lib/db/url.ts` builds the connection string: an explicit `DATABASE_URL` wins, otherwise it is composed from `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER` and `DB_PASSWORD` (Section 6).
Kysely and `db:migrate` NEW	Kysely is the type-safe query builder; `kysely-ctl` runs the migrations. `npm run db:migrate` brings any database up to date (Sections 7 and 8).
Reference data in migrations NEW	The 4 routes, the 14 default rules, the status mapping and the Genesys regions are written by migrations, so a freshly migrated database is ready to use with no separate seed step.
Storage layer NEW	A `Store` interface with two implementations: in memory (development, tests, replays) and PostgreSQL. `PERSISTENCE_MODE` picks one; memory is refused outside development and test (Section 10).
Rules survive a restart CHANGE	`registry.rules` in `lib/server/runtime.ts` is loaded from the database instead of `defaultRules()`, and every rule edit writes a new `rule_version`.
Write-behind queue CHANGE	The engine stays in charge, in memory. What it records is queued and written to PostgreSQL in batches, so the one-second tick never waits on the database (Section 10).
Shared RDS instance NEW	A new database, `rtm_console`, on the existing `van-reservation-prod` instance: no added cost, with its own roles and capped connections so it cannot touch or slow down Van Reservation (Section 4).
Local PostgreSQL NEW	`docker-compose.yml` runs a pinned PostgreSQL on port 5435, with the same names as `.env.example` (Section 11).
Partitions never missing FIX	`notification_log`, partitioned by month, gets a default partition and the next three months, so an insert can never fail for want of a partition (Section 9).
Fewer writes for strike state FIX	`agent_rule_state` stores strikes only, not the `fired` flag that flips at every status change (Section 9).
Live floor only FIX	Only the Gencloud floor is saved. CSV replays stay in memory, and the simulator is saved only when asked for, so simulated call-outs never mix with real ones in reports (Section 9).

Change	What it means
Call-outs, incidents, strikes WAITS	Phase 2: both questions are answered (Section 1); waits on real IDs and the Phase 1 tables (Section 12).
Raw event log NEW	Phase 3: holds today's Genesys events only, so the KPIs survive a restart. Emptied at midnight Eastern (Section 9).

3 What gets saved, and when

The console keeps working the way it does today: it checks the floor every second and updates the screen straight away. The database is its record and its memory across restarts. Every time something happens on the floor or someone acts in the console, a record is also saved.

How a record is saved

1. **The console acts first.** The call-out, acknowledgement or rule change shows on screen at once, as today.
2. **A record joins a queue.** Records wait in the order they happened.
3. **About once a second, the queue is written in one go.** Either the whole batch is saved or none of it is; a failed write is tried again with the same records in the same order. If the database is down, the floor keeps running and records wait in the queue (Section 10).

Most events **add a new row** (a call-out, a rule version, an audit entry). Actions on something that already exists, such as acknowledging a call-out or closing an incident, **update its row**, so it always shows its latest state. Nothing is deleted.

Scenarios

When	What is saved	Table	From
The database is first set up (`db:migrate`)	New rows: the 14 default rules, the 4 alert routes and their steps, the Genesys status names and regions. Once, so a new database is ready to use.	`rules_master`, `rule_version`, `notification_route`, `notification_route_step`, `status_mapping`, `genesys_region`	Phase 0
An admin changes a rule: threshold, severity, route, or on/off	A new version of the rule, with who changed it and when. The rule then points at that version. Older versions are kept, so every call-out shows the rule as it was when it fired. Quick edits in a row become one version.	`rule_version`, `rules_master`	Phase 0
The app starts	Nothing is written. It reads the rules and, from Phase 2, today's shift, strike counts, open call-outs and open incidents, so it carries on where it stopped.	(reads only)	Phase 0
Agents, teams and queues come in from Genesys	New rows for new people, teams and queues; updated rows when a name, team or queue membership changes.	`person`, `team`, `queue`, `queue_member`	Phase 1
Someone turns on desktop alerts	A new row for that browser, removed from use when they turn alerts off.	`push_subscription`	Phase 1

When	What is saved	Table	From
A new day starts (midnight, US Eastern)	A new shift row for that Eastern date. Every call-out and incident that day links to it, and strike counts start again at zero. From Phase 3, the raw events table is emptied, so the KPIs start again at zero. That is the only data removed: open call-outs and incidents carry over and stay on screen until someone acknowledges or closes them.	`shift`	Phase 2
An agent breaks a rule (strikes 1 and 2)	A new call-out row: agent, rule and its version, the value that broke it, the strike number and who it went to (team lead, then lead). The agent's strike count for that rule is updated.	`monitoring_instance`, `agent_rule_state`	Phase 2
Third strike	As above, plus a new incident with the next incident number (for example INC-2026-0042), saved together with the call-out so neither exists without the other.	+ `incident_log`, `incident_counter`	Phase 2
Fourth strike and later	The same as the third strike: a new call-out row and a **new** incident with the next number, saved together. Strike 5 means 3 incidents. Earlier incidents are not changed.	`monitoring_instance`, `incident_log`, `incident_counter`	Phase 2
A floor-wide call-out (queue numbers, no single agent)	A new call-out row with no agent and no strike.	`monitoring_instance`	Phase 2
An alert is sent (desktop or in the console)	A new row: who it went to, when, and whether it was delivered.	`notification_log`	Phase 2
A team lead acknowledges a call-out	The call-out row is updated: acknowledged, and when.	`monitoring_instance`	Phase 2
Acknowledge all	Each open call-out in view is updated as above, plus one audit entry for the bulk action.	`monitoring_instance`, `audit_event`	Phase 2
A reason is added to a call-out	The call-out row is updated with the reason (and acknowledged, if sent from the nudge).	`monitoring_instance`	Phase 2
An incident moves: Open, Investigating, Closed	The incident row is updated with its status, outcome and closing time, plus an audit entry for each move.	`incident_log`, `audit_event`	Phase 2

When	What is saved	Table	From
Every raw Genesys update	New rows, many per write, about once a second. The table only ever holds the current Eastern day; on start-up the console rebuilds the day's KPIs from it.	`source_event_stream`	Phase 3

NOT SAVED

CSV replays and simulator runs stay in memory, so practice data never mixes with real call-outs. Live timers (time in state, hold time) are not saved either: Genesys sends them again when the console reconnects. The Genesys token is a secret and never goes in a table.

Example: one agent's afternoon

An agent goes over the hold-time limit several times. This is what the database holds after each step.

Time	What happens	What is saved
14:05	Over the limit: strike 1, call-out to the team lead.	Call-out #1 added (strike 1, to team lead, open). Strike count set to 1.
14:07	The team lead acknowledges and adds a reason.	Call-out #1 updated: acknowledged at 14:07, with the reason.
15:20	Over the limit again: strike 2, to the lead.	Call-out #2 added (strike 2, to lead). Strike count set to 2.
16:10	Strike 3: an incident is opened and the manager is notified.	Call-out #3 added, strike count set to 3, incident INC-2026-0042 added (Open), all in one write. Alert to the manager logged.
16:30	Operations starts investigating.	INC-2026-0042 updated to Investigating; audit entry added.
17:00	Operations closes it as coached.	INC-2026-0042 updated to Closed, outcome "Coached", closed at 17:00; audit entry added.
17:05	The console restarts for an update.	Nothing lost. On start it reads strike count 3 and the closed incident, and carries on.

How it fits together



While running, the engine writes forward. At start-up the arrow reverses: the engine loads rules, strikes, open call-outs and incidents from PostgreSQL.

Three rules for the design

Rule	Why
The engine never waits on the database	A slow query or a failover would freeze every timer on the floor. Writes go into a queue and are flushed together, about once a second.
One engine process	The engine's state is in memory, so two copies behind a load balancer would disagree. The console runs as **one** long-lived task. With one writer, two strikes can never be numbered at the same time; a unique key on the incident's call-out stays as a safety net.
The database enforces the business rules	One incident per strike from 3 on, call-outs pointing at real rules and people, and the version of a rule that fired, are all checked by keys and indexes, not by our code.

For developers: where the engine writes today

These are the functions behind the scenarios above. Each one will hand a record to the store as well as changing memory.

Code	Becomes
`fire()` , `lib/engine/escalation.ts`	A `monitoring_instance` row and a strike in `agent_rule_state`
`openIncident()` , same file	An `incident_log` row numbered from `incident_counter` , in the same transaction as the call-out
`ack` , `ackAll` , `comment` , `lib/engine/engine.ts`	Updates to `monitoring_instance` ; `ackAll` also writes an `audit_event`
`invAction`	An `incident_log` update and an `audit_event`
`setThr` , `setSev` , `setRoute` , `setOn`	A new `rule_version` , and `rules_master.current_version` moved to it
`pushBatch()` , saving a subscription, `lib/server/push.ts`	`notification_log` and `push_subscription` rows
`engine.reset()` in `createGencloud()`	Opening or resuming the active `shift`

4 Hosting on AWS

For now the console reuses what is already running: the app on an EC2 instance, and its data in a **new database on the existing RDS instance** `van-reservation-prod` . Same instance, so no added cost. The console can move to an instance of its own later by changing one connection string.

Piece	Choice	Notes
App	An EC2 instance, **one Node process**	Most likely the current host; to be confirmed. Runs `next start` under a process manager (systemd or pm2) that restarts it if it stops. Not Lambda or Amplify server rendering: they stop the process between requests.

Piece	Choice	Notes
Database	A new database, `rtm_console`, on the RDS instance `van-reservation-prod`	Its own database, not a schema inside the Van Reservation one, with its own roles (Section 6). Backups, maintenance window and PostgreSQL version are the instance's.
Secrets	AWS Secrets Manager	One secret per RTM database role, plus the Genesys and VAPID secrets. Read at start-up through the EC2 instance profile, or written to an environment file only root can read.
Network	Security groups	The RDS security group already admits the Van Reservation app. Add the RTM EC2 instance's security group on port 5432, and nothing else.
Logs	The process manager, or CloudWatch Logs	A failed start-up check (Section 5) appears as one readable message.

⚠ SHARING A PRODUCTION INSTANCE: GROUND RULES

Van Reservation is live on this instance. The console must never be able to read its data or slow it down.

- **Separate database, separate roles.** `rtm_migrator` and `rtm_app` can connect to `rtm_console` only; `CONNECT` on it is revoked from `PUBLIC`, and the RTM roles are granted nothing on the Van Reservation database (see the note after the setup steps below).
- **Capped connections.** Each RTM role gets a `CONNECTION LIMIT` (for example 15 for the app, 3 for the migrator), so the console can never use up the instance's connection slots.
- **No instance-wide changes.** No parameter-group edits, no extensions that need `shared_preload_libraries`, no reboots for RTM's sake. Anything like that waits for a dedicated instance.
- **Only deployed environments connect.** Developers work against the local Docker database (Section 11), never against this instance.
- **Batched writes.** The write-behind queue (Section 10) keeps the console's load to one small transaction about once a second.

Creating the database and roles in pgAdmin (once)

This is done once, by someone with the master login for `van-reservation-prod`, in pgAdmin. It creates only RTM objects: two roles, one database and its grants. Nothing on the Van Reservation database changes. From then on, `db:migrate` creates and changes the tables.

Object	Type	Settings
`rtm_migrator`	Login role	Connection limit 3. Owns `rtm_console`. Used only by `db:migrate`.
`rtm_app`	Login role	Connection limit 15. Reads and writes rows; cannot create or change tables.
`rtm_console`	Database	Owner `rtm_migrator`. `CONNECT` for the two RTM roles only.

STATUS: STEPS 1 TO 7 DONE ON 2026-11-03

The setup below was run on `van-reservation-prod` (PostgreSQL 17.7) as its master login, `vanadmin`, through pgAdmin. The steps are written as they worked there; step 8, the test login as `rtm_app`, is still to do. Keep them for a rebuild or a move to another instance.

BEFORE YOU START IN PGADMIN

- **Run one step at a time.** In the Query Tool, highlight the statements for a step and press **F5**: pgAdmin then runs only the highlighted text. Step 4 (`CREATE DATABASE`) must run on its own; PostgreSQL refuses it when it is sent together with other statements.
- **pgAdmin shows only the last result.** When several statements run together, *Data Output* shows the last one's rows and *Messages* the last one's message. Run a check query on its own to see its result.
- **The master login is not the owner.** On RDS the master login is not a superuser, and since PostgreSQL 16 it does not take on the rights of a role it created. A `GRANT` or `REVOKE` it runs on `rtm_console` is skipped with only a warning, not an error. So steps 5 and 6 first switch to the owner with `SET ROLE rtm_migrator`, and end with `RESET ROLE`.
- **Passwords never go in the Query Tool as plain text.** The role dialog's password field is locked for the RDS master login, so step 3 sets each password from a hash made on your own PC. Neither pgAdmin's history nor the server's logs ever hold the password itself.
- **Check the server name** in the Query Tool's title bar before each step: it must be `van-reservation-prod`, and the database must be the one the step names. If pgAdmin reaches the instance through a tunnel, the address shows as `127.0.0.1` with a local port; that is expected.

1. **Open a Query Tool on the `postgres` database.** In the Object Explorer, expand the `van-reservation-prod` server, then *Databases*, right-click `postgres` and choose *Query Tool*. Roles belong to the whole instance, so this database is the right place for steps 1 to 5, and it keeps you out of the Van Reservation database. First, record what the instance is running:

```
SELECT version();          -- the PostgreSQL version: the local Docker image must match it (Section 11)
SELECT current_user;       -- the master login (vanadmin); see the check below
SELECT datname FROM pg_database WHERE NOT datistemplate;   -- existing databases; rtm_console must not
be listed yet
```

   Run each line on its own to see its result. To confirm the login is the instance's master login, all three columns must be *true*:

```
SELECT pg_has_role(current_user, 'rds_superuser', 'MEMBER') AS is_rds_master,
       rolcreaterole AS can_create_roles, rolcreatedb AS can_create_databases
FROM pg_roles WHERE rolname = current_user;
```

2. **Create the two roles.** They are created without passwords, so nobody can log in as them until step 3. The `GRANT` lets the master login create a database owned by `rtm_migrator` (step 4) and switch to it with `SET ROLE` (steps 5 and 6).

```
CREATE ROLE rtm_migrator LOGIN CONNECTION LIMIT 3;
CREATE ROLE rtm_app      LOGIN CONNECTION LIMIT 15;
GRANT rtm_migrator TO CURRENT_USER;
```

3. **Set the passwords from a hash.** On your own PC, in a terminal of your own (PowerShell or the VS Code terminal; **not** through an AI assistant or anything else that records the output), in the repository:

```
npm run db:role-passwords
```

It prints a new random 40-character password for each role, once, and an `ALTER ROLE ... PASSWORD 'SCRAM-SHA-256$...'` line for each. Save both passwords in a password manager straight away, then in AWS Secrets Manager (one secret per role), and never in a file, chat or ticket. Paste only the two `ALTER ROLE` lines into the Query Tool on `postgres` and run them; then close the terminal. If a password is ever seen by anyone else, run the command again and apply the new lines: the old password stops working.

4. **Create the database**, highlighting this statement on its own:

```
CREATE DATABASE rtm_console OWNER rtm_migrator;
```

5. **Close the database to everyone else.** By default every role may connect to a new database; this limits `rtm_console` to its owner, `rtm_app` and the master login. The grants run as the owner (see *Before you start*).

```
SET ROLE rtm_migrator;
REVOKE CONNECT, TEMPORARY ON DATABASE rtm_console FROM PUBLIC;
GRANT CONNECT ON DATABASE rtm_console TO rtm_app;
GRANT CONNECT ON DATABASE rtm_console TO vanadmin;    -- the master login from step 1
RESET ROLE;
```

Then check, on its own:

```
SELECT current_user AS back_to,                                          -- vanadmin
       has_database_privilege('vanadmin', 'rtm_console', 'CONNECT') AS master_can_connect,   -- true
       has_database_privilege('rtm_app',  'rtm_console', 'CONNECT') AS app_can_connect,      -- true
       has_database_privilege('public',   'rtm_console', 'CONNECT') AS everyone_can_connect; -- false
```

If pgAdmin then says *permission denied for database "rtm_console"* when you open it, it is reusing an attempt made before the grants: close any tab on `rtm_console`, then *Disconnect Server* and *Connect Server*.

6. **Set the grants inside `rtm_console`.** Right-click *Databases*, *Refresh*, then right-click `rtm_console` and open a **new** Query Tool on it: these grants apply only to the database the Query Tool is connected to. The default privileges give `rtm_app` read and write access to every table and sequence that `rtm_migrator` creates later, so no migration has to grant them one by one. They are set while acting as `rtm_migrator`, so they apply to the objects it creates.

```
SET ROLE rtm_migrator;
REVOKE CREATE ON SCHEMA public FROM PUBLIC;    -- already the default on PostgreSQL 15 and later
GRANT USAGE, CREATE ON SCHEMA public TO rtm_migrator;
GRANT USAGE ON SCHEMA public TO rtm_app;
ALTER DEFAULT PRIVILEGES IN SCHEMA public
  GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO rtm_app;
ALTER DEFAULT PRIVILEGES IN SCHEMA public
  GRANT USAGE, SELECT ON SEQUENCES TO rtm_app;
RESET ROLE;
```

Then check, on its own, in the same Query Tool:

```
SELECT current_user AS me,                                                    -- vanadmin
       has_schema_privilege('rtm_migrator', 'public', 'CREATE') AS migrator_can_create, -- true
       has_schema_privilege('rtm_app', 'public', 'USAGE')        AS app_can_use,         -- true
       has_schema_privilege('rtm_app', 'public', 'CREATE')       AS app_can_create,      -- false
       has_schema_privilege('public',  'public', 'CREATE')       AS everyone_can_create, -- false
       (SELECT string_agg(defaclobjtype::text || ': ' || defaclacl::text, '  ')
          FROM pg_default_acl WHERE defaclrole = 'rtm_migrator'::regrole) AS future_grants;
```

`future_grants` must read `r: {rtm_app=arwd/rtm_migrator} S: {rtm_app=rU/rtm_migrator}`: tables readable and writable by `rtm_app`, sequences usable by it. If a statement fails partway, close the tab without saving, so the session does not stay switched to `rtm_migrator`.

7. **Check the roles.** In the Query Tool on `postgres`:

```
SELECT rolname, rolcanlogin, rolconnlimit
FROM pg_roles WHERE rolname LIKE 'rtm\_%' ORDER BY rolname;  -- rtm_app 15, rtm_migrator 3
```

The owner and who may connect were checked in step 5.

8. **Test the app login.** This is the only step that proves the `rtm_app` password works. In pgAdmin, *Object → Register → Server*: a name such as *rtm-test (delete me)*; on *Connection*, the same host and port as `van-reservation-prod` (and the same *SSH Tunnel* settings, if it uses one); maintenance database `rtm_console`, user `rtm_app`, and **Save password** left off; SSL mode *require* unless the tunnel already protects the link. Paste the password when asked. In a Query Tool on that connection, one line at a time:

```
SELECT current_user, current_database();    -- rtm_app, rtm_console
CREATE TABLE should_fail (x int);           -- must fail: permission denied for schema public
```

Then remove the test server registration. A login from your PC only works through the tunnel or if the RDS security group admits your address; the deployed app connects from the EC2 instance.

THE VAN RESERVATION DATABASE

PostgreSQL lets every role connect to an existing database unless `CONNECT` was revoked from `PUBLIC` there, so `rtm_app` may be able to open the Van Reservation database. It still cannot read any of its tables unless they were granted to `PUBLIC`. To confirm, open a Query Tool on the Van Reservation database and run:

```
SELECT table_schema, table_name, privilege_type
FROM information_schema.role_table_grants
WHERE grantee = 'PUBLIC'
  AND table_schema NOT IN ('pg_catalog', 'information_schema');   -- expect no rows
```

Revoking `CONNECT` from `PUBLIC` on the Van Reservation database would close this fully, but it is a change to Van Reservation: do it only with its owner's agreement, after checking which roles its app uses.

UNDOING IT

Until the console has stored real data, the whole setup can be removed without touching anything else. Close every tab and connection on `rtm_console` first. Then, in the Query Tool on `postgres`, run each line on its own, in this order (only the owner may drop the database, and `DROP DATABASE` refuses to run together with other statements):

```
SET ROLE rtm_migrator;
DROP DATABASE rtm_console;
RESET ROLE;
DROP ROLE rtm_app;
DROP ROLE rtm_migrator;
```

Moving out later

When the console needs its own instance (more load, its own maintenance window, an extension that needs a parameter change), dump `rtm_console` with `pg_dump`, restore it on the new instance, and change `DATABASE_URL`. Nothing in the code depends on sharing.

AWS CREDENTIALS IN THE APP

On EC2 the app uses the **instance profile** through the AWS SDK's default credential chain: no access keys in the environment. Explicit keys are accepted only for local work, and the environment check refuses half a set: a key without its secret, or a temporary `ASIA…` key without its session token.

5 Configuration: one validated environment

Today eight files read `process.env` directly, and a missing value shows up only when a feature first needs it. After this change the application reads one parsed, typed object, and a broken deployment fails at start-up with a message that names every problem.

How it works

1. `lib/env.ts` exports `parseEnv(raw)`, a pure function that checks a set of variables against a Zod schema and returns a typed object. It is what the tests call.
2. The same file exports `env()`, which parses `process.env` once and caches the result. Everything else in the app imports `env()`; nothing else reads `process.env`.
3. `instrumentation.ts` calls `env()` in `register()`, on the Node.js runtime. If anything is wrong the server stops before it accepts a request, and the log lists **every** failing variable with its reason, not just the first.

The rules every variable follows

Rule	In practice
No silent default for a secret	Passwords, tokens and keys are required where they are needed and never filled in for you.
Empty is not missing	`KEY=` sets an empty string, which fails validation. A commented-out `# KEY=` means the variable is absent, which is allowed for optional settings.
Safe values may default	For example `DB_PORT` (5432), `DB_POOL_MAX` (10), `RTM_TIME_ZONE` (America/New_York).
Pairs are checked together	Both Genesys OAuth values or neither; all three VAPID values or none; `PERSISTENCE_MODE=postgres` requires database settings. These checks live in the schema, not in the code that uses them.
Stubs only where safe	`PERSISTENCE_MODE=memory` is accepted only when `APP_ENV` is `development` or `test`. It is an allowlist, so a new environment name is refused by default.

WHY APP_ENV AND NOT NODE_ENV

Every `next build` runs with `NODE_ENV=production`, including staging and a developer's production build. `APP_ENV` says which environment the server is really in.

New and existing variables

Variable	Required	What it does
`APP_ENV` NEW	Yes	`development`, `test`, `staging` or `production`.
`PERSISTENCE_MODE` NEW	Yes	`postgres` or `memory` (development and test only).
`DATABASE_URL` NEW	Either this…	Full connection string. Always wins over the `DB_*` values.

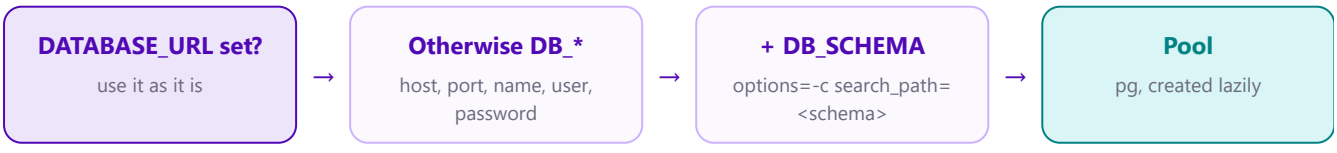
Variable	Required	What it does
`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD` NEW	…or these	Composed into a URL when `DATABASE_URL` is not set. The password is URL-encoded for you.
`DB_SCHEMA` NEW	No	A PostgreSQL schema to work in, set through `search_path`. Lets several developers share one database.
`DB_SSLMODE` NEW	No	`verify-full` by default. `disable` for the local container; `no-verify` refused outside development.
`DB_CA_CERT` NEW	No	Path to an extra certificate authority, for PCs behind corporate TLS inspection. Added to the trusted list, never a replacement for it.
`DB_POOL_MAX` NEW	No	Connections in the pool; default 10.
`TEST_DATABASE_URL` NEW	No	Database for integration tests (Section 11).
`NEXT_PUBLIC_FEED`, `NEXT_PUBLIC_VIEW_AS`, `NEXT_PUBLIC_DEV_TOOLS`	As today	Read at build time and inlined into the browser bundle, so they are checked at build as well.
`RTM_SITE_PASSWORD`, `RTM_ADMIN_SECRET` CHANGE	As today	Now also given a minimum length, and known placeholder values are refused. Unset still means "page locked"; set but wrong still means "unauthorized".
`GENESYS_TOKEN`, `GENCLOUD_API_BASE`, `RTM_VIEW_CONFIG_ID`, `GENCLOUD_CLIENT_ID`, `GENCLOUD_CLIENT_SECRET`	For the live feed	Unchanged in meaning; read through `env()`.
`VAPID_PUBLIC_KEY`, `VAPID_PRIVATE_KEY`, `VAPID_SUBJECT`	All or none	Desktop alerts, as today.

`.env.example` is the contract: every variable carries a comment saying what it does, whether it is required, what breaks if it is wrong and how to keep it safe. Copied as it is, it must fail the start-up check rather than run with an unsafe value.

6 Connecting to PostgreSQL

One connection pool per process, created only when the first query needs it. The URL logic is a small pure module with its own tests, kept apart from Next.js.

How the URL is chosen



A `DATABASE_URL` exported in the shell beats the one in `.env.local`: the loader never overwrites a variable that is already set.

`schemaFromUrl(url)` reads the schema a URL points at from its `search_path` option. The tests use it to make sure they never run against the main schema.

Two roles, least privilege

Role	Can	Used by
`rtm_migrator`	Owns the schema: create, alter and drop tables, indexes and partitions.	`db:migrate` only, in the deploy script.
`rtm_app`	Select, insert, update and delete rows. Cannot change the schema. Granted through default privileges, so new tables are covered automatically.	The running console.

If the app is ever compromised it cannot drop a table, and a typo in app code cannot alter the schema.

Encryption in transit

- Whether the instance insists on TLS is set in its parameter group, which belongs to Van Reservation. The console uses TLS either way.
- The app verifies the server against the **RDS certificate bundle**, kept on the EC2 instance with the app (`DB_SSLMODE=verify-full`).
- On a PC behind corporate TLS inspection, `DB_CA_CERT` adds the company's certificate to the trusted list. Turning verification off is never the fix.

> **WHY PG IS KEPT OUT OF THE BUNDLE**
>
> `pg` is added to `serverExternalPackages` in `next.config.ts` , next to `exceljs` and `web-push` , so Next.js loads it from `node_modules` at run time instead of bundling it.

7 Migrations with Kysely

Kysely is a type-safe SQL query builder, not an ORM: queries read like SQL and are checked by TypeScript. Its migrator applies numbered TypeScript files in order and records each one in the database. `kysely-ctl` is its command-line tool.

The commands

Command	Does	Needs the database
`npm run db:migrate`	Applies every migration not yet applied, oldest first. Does nothing when up to date.	Yes, as `rtm_migrator`
`npm run db:make -- <name>`	Creates an empty, timestamped migration file to fill in.	No
`npm run db:status`	Lists the migrations and which are applied.	Yes
`npm run db:rollback`	Undoes the last migration. **Local only**: refused when `APP_ENV` is staging or production.	Yes
`npm run db:types`	Regenerates `lib/db/schema.ts`, the TypeScript description of every table, from a migrated database (`kysely-codegen`).	Yes, local
`npm run db:role-passwords`	Prints a new random password for each RTM database role, once, and the `ALTER ROLE` line that sets it from a SCRAM hash, so the password itself never reaches pgAdmin (Section 4, step 3). Run it in a terminal of your own.	No
`npm run db:up`	Starts the local PostgreSQL container.	No

Files

```
# configuration, read by kysely-ctl; uses lib/env.ts for the connection
kysely.config.ts
# one file per change, applied in name order
lib/db/migrations/
  <timestamp>_reference.ts            # Phase 0: regions, orgs, routes, rules, status mapping, audit
  <timestamp>_people_and_queues.ts    # Phase 1
  <timestamp>_shift_records.ts        # Phase 2
lib/db/schema.ts                      # generated by db:types, committed
lib/db/client.ts                      # the lazily created Kysely instance
lib/db/url.ts                         # DATABASE_URL composition, schemaFromUrl()
lib/store/                            # the Store interface, memory and PostgreSQL
tools/db/                             # db:types, the rollback guard, the test-database init script, role
passwords
```

The TypeScript scripts in `tools/db/` run with `tsx`, a dev dependency; `role-passwords.mjs` runs with plain Node.

A migration, in outline

```
import { type Kysely, sql } from "kysely";

export async function up(db: Kysely<unknown>): Promise<void> {
  await db.schema.createTable("notification_route")
    .addColumn("route_id", "text", c => c.primaryKey())
    .addColumn("route_name", "text", c => c.notNull())
    .addColumn("enabled", "boolean", c => c.notNull().defaultTo(true))
    .execute();
  // ...then insert the four routes
}

export async function down(db: Kysely<unknown>): Promise<void> {
  await db.schema.dropTable("notification_route").execute();
}
```

The schema builder covers almost all of the design: identity columns, generated columns, check constraints, partial indexes and deferrable foreign keys. The rest, such as `PARTITION BY RANGE`, is written with Kysely's `sql` template inside the same migration.

The migrations, by phase

What each migration creates, with the decisions in Section 1 built in. `db:make` adds a timestamp to each file name, so they always run in this order.

Migration	Creates	Built in from the decisions
`reference` Phase 0	`genesys_region`, `genesys_org`, `notification_route`, `notification_route_step`, `rules_master`, `rule_version`, `status_mapping`, `audit_event`, with their rows	The three Genesys orgs the console watches and their regions; the 14 default rules as version 1; the 4 routes. `changed_by` and `updated_by` have no foreign key yet.
`people` Phase 1	`team`, `person`, `queue`, `queue_member`, `push_subscription`	People keyed by Genesys user ID, each with their org. Adds the `person` foreign keys left out in Phase 0.
`shifts` Phase 2	`shift`, `agent_rule_state`	A shift is **one US Eastern day per org**: `org_id` and `shift_date`, unique together, starting and ending at Eastern midnight. One active shift per org. Strike rows are per shift, so strikes start at zero each day with nothing deleted. `agent_rule_state` keeps `strikes` only, not `fired`.

Migration	Creates	Built in from the decisions
`callouts_and_incidents` Phase 2	`monitoring_instance`, `incident_counter`, `incident_log`, `notification_log`	**One incident per strike** from 3 on: `incident_log` gets `instance_id`, the call-out that opened it, unique, and `strike` (3 or more). No `one_open_incident` index. **One incident counter for every org**, one row per year, so numbers start again at 0001 each January. Call-outs and incidents carry over midnight. `source` no longer allows `csv`: replays are never saved. `notification_log` is split by month, with a default partition and three months ahead.
`events` Phase 3	`source_event_stream`	**Holds the current Eastern day only**, for the KPIs. Not partitioned; emptied with `TRUNCATE` at midnight Eastern, for every org at once. Each event carries its `org_id`. Grants `TRUNCATE` on this table to `rtm_app`.

Rules for writing migrations

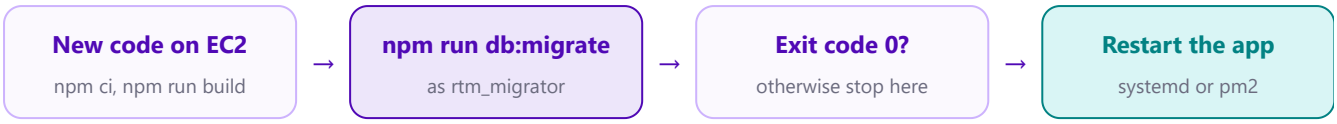
Rule	Why
Never edit a migration that has run outside your machine	It will not run again anywhere it already ran. Write a new one.
Production goes forward only	`down()` is for local work. A bad change in production is fixed by a new migration.
Reference data belongs in migrations	Routes, default rules, status mapping and regions are written once, by the migration that creates their table. A later rule edit by an Admin is never overwritten.
One concern per file	A failed migration is rolled back as a whole and is easy to read in review.
Regenerate types and commit them	After a new migration, run `db:types` and commit `lib/db/schema.ts` with it, so queries are checked against the real columns.

SAFE TO RUN TWICE, SAFE TO RUN TOGETHER

Kysely records applied migrations in `kysely_migration` and holds a lock in `kysely_migration_lock` while it works. Running `db:migrate` again does nothing, and two runs at once wait for each other. It also refuses to apply an older file after a newer one has run, which catches a branch merged out of order.

8 Running db:migrate before a deployment

The RDS instance has no public address, so `db:migrate` has to run from inside the VPC: from a laptop it cannot connect. The EC2 instance that runs the console is inside the VPC, so the deployment runs it there, after the new code is built and before the app is restarted.



The new code starts only after the database has been migrated for it.

Step by step

1. On the EC2 instance, get the new code into a **new release folder** and run `npm ci` and `npm run build` there. The running version keeps serving the floor from its own folder; building over it would replace the `.next` folder it is using.
2. Run `npm run db:migrate` with the `rtm_migrator` credentials, set for that one command only (for example read from Secrets Manager into `DATABASE_URL`). The app's own environment holds only `rtm_app`.
3. **Exit code 0:** point the process manager at the new release folder and restart the console. On start-up it validates its settings and loads its rules from the new schema. The previous folder stays for a quick rollback of the code.
4. **Anything else:** stop. Do not restart: the running console is untouched, because a failed migration is rolled back inside its transaction. Read the error, fix it with a new commit, and deploy again.

These steps go into a deploy script in the repository (for example `scripts/deploy.sh`), so a deployment is one command and the migration can never be skipped.

THE MIGRATOR TOUCHES ONLY RTM_CONSOLE

`rtm_migrator` owns `rtm_console` and has no rights on the Van Reservation database, so a migration cannot change anything but the console's own tables, however it is written.

WRITE MIGRATIONS THE OLD CODE CAN LIVE WITH

For a short time the old app runs against the new schema. Add columns and tables freely. To rename or remove one, use two releases: first stop using it, then drop it.

Running it by hand

To run a migration from a laptop, for example to check a new one against a copy of the data, open an SSM port-forwarding session to the database through the EC2 instance, point `DATABASE_URL` at the forwarded local port with the migrator's credentials, and run `npm run db:migrate`. Keep this for emergencies; the deploy script is the normal route.

9 Changes to the proposed schema

The 20 tables in the PostgreSQL Schema doc are kept. These are the corrections found in review, each to match how the engine actually runs. Each one is made in the first migration of its table.

Area	In the schema doc	Change
Strike 3 and later FIX	The incident number is taken from `incident_counter` inside the transaction; a race is retried.	The engine numbers incidents in memory, from one counter shared by every org (`registry.incidentSeq`), and cannot wait mid-tick. At start-up the counter continues from `incident_counter` ; the flush writes the counter, incident and call-out in one transaction. With one engine process there is no race.
Data sources FIX	`shift.source` allows gencloud and sim; `monitoring_instance.source` also allows csv. Sim agents have no Genesys ID or org.	Only the Gencloud floor is saved. CSV replays are private to a session and stay in memory. The simulator is saved only behind a setting, under its own org and people.
Partitions FIX	`notification_log` and `source_event_stream` are partitioned by month.	`notification_log` stays monthly, with a `DEFAULT` partition and the next three months at creation, and a monthly job keeping three months ahead. Not `pg_partman` for now: it would mean changing the shared instance's parameter group.
Raw events FIX	`source_event_stream` is kept by month.	It holds **one day**. It is not partitioned: at midnight Eastern it is emptied with `TRUNCATE` , which is instant and gives the space back at once, unlike deleting rows. The emptying goes through the same ordered write queue as the events, so an event from 23:59:59 can never land after it. The migration grants `TRUNCATE` on this one table to `rtm_app` .
Orgs FIX	`shift` has no org; `shift_id` is a name such as `'2026-10-07-day'` , with one active shift per source.	The console runs one floor per Genesys org. `shift` gets `org_id` and `shift_date` (the Eastern date), unique together, and one active shift **per org**. Call-outs, incidents and strikes reach their org through the shift; raw events carry `org_id` themselves.
Incident numbers FIX	`incident_counter` has one row per year.	Kept, and **shared by every org**, so numbers never repeat across orgs. Numbering starts again at `0001` each January (on the Eastern date): `INC-2027-0001` follows `INC-2026-…` . The engine already does this (#37): one shared counter that keeps its year and last number (`{ year, n }`), the same pair this table stores. At start-up it continues from this table.
Strike state FIX	`agent_rule_state` stores `strikes` and `fired` .	Store `strikes` only. `fired` flips at every status change for about 1,600 agents. After a restart the engine already marks rules an agent is past as fired, without a call-out.

Area	In the schema doc	Change
Event volume FIX	Tens of thousands of raw events per shift.	Plan for hundreds of thousands in a day. Events are buffered and inserted many rows per statement, once a second. The table never holds more than one day.
Times FIX	`timestamptz` everywhere.	Kept. Every record is stamped with the real date and time at the moment it happens. The engine's seconds since midnight only drive the screen and are not stored.
Incidents FIX	One open incident per agent per rule (`one_open_incident`); later strikes raise its count.	**One incident per strike** from strike 3 on. The `one_open_incident` index is dropped; instead each incident points at the call-out that opened it, and that link is unique, so one strike can never open two incidents. Each incident stores its strike number. The engine's `openIncident()` changes to match: today it reuses an open incident; it will always open a new one.
Rule edits	Every edit is a new `rule_version` .	Kept. Edits to the same rule by the same person within a few seconds are folded into one version, so typing a threshold does not leave a version per keystroke.

Tables by phase

Phase	Tables	Status
0 · Reference	`genesys_region` , `genesys_org` , `notification_route` , `notification_route_step` , `rules_master` , `rule_version` , `status_mapping` , `audit_event`	Can start
1 · People	`person` , `team` , `queue` , `queue_member` , `push_subscription`	Needs real IDs
2 · Shift records	`shift` , `agent_rule_state` , `monitoring_instance` , `incident_log` , `incident_counter` , `notification_log`	After Phase 1
3 · Raw events	`source_event_stream`	After Phase 2

`rule_version.changed_by` and `status_mapping.updated_by` point at `person` . In Phase 0 they are created without that foreign key, which Phase 1 adds once `person` exists.

10 How the engine saves and recovers

The engine keeps working exactly as it does today. What changes is that every record it makes is also handed to a store, and that on start-up it begins from what the store remembers instead of from nothing.

The store

MEMORY STORE

Keeps nothing beyond the process. Used in development without a database, in unit tests, and always for CSV replays. Refused in staging and production.

POSTGRESQL STORE

Queues each record in order and flushes the queue in one transaction about once a second. A failed flush is retried with the same records in the same order; the console raises a notice if the queue keeps growing.

On start-up

1. Validate the environment (Section 5). Stop if anything is wrong.
2. Load the rules at their current version. In Phase 0 this is the only step.
3. From Phase 2: find or open the shift for today's Eastern date, then load today's strike counts, every open call-out and incident from any day, and the incident counter.
4. From Phase 3: rebuild today's KPIs from the events in `source_event_stream`.
5. Connect the feed. Genesys gives each agent's time already in state; rules an agent is past are marked fired without a call-out, as today.

What is never stored

Data	Why not
Live timers (time in state, hold time)	Taken again from Genesys at connect.
The `fired` flag per rule episode	Rebuilt at connect (Section 9).
CSV replays	Private to the session that started them.
The Genesys token	A secret: it stays in the environment or Secrets Manager, never in a table.

IF THE DATABASE IS DOWN

The floor keeps running: timers, call-outs and alerts carry on from memory, and records wait in the queue. A start-up with the database unreachable fails, because the engine cannot know the current rules or strikes.

11 Local development and tests

A developer can work with no database at all, as today, or against a local PostgreSQL in Docker that behaves like RDS. Developers never connect to `van-reservation-prod`.

First run with a local database

1. Copy `.env.example` to `.env.local` and set `APP_ENV=development`, `PERSISTENCE_MODE=postgres` and the `DB_*` values given for the container.
2. `npm run db:up` starts PostgreSQL in Docker on **port 5435**, so it never clashes with a PostgreSQL already installed on 5432 or with other local containers on 5433 and 5434.
3. `npm run db:migrate` creates every table and the reference data.
4. `npm run dev` as usual. Rule edits now survive a restart.

To work without a database, set `PERSISTENCE_MODE=memory`. Nothing else changes.

The container

Setting	Value
Image	A pinned `postgres` release with the same major version as `van-reservation-prod`
Container name	`rtm-console-db`
Host port	5435
Data	A named volume, `rtm-console-pgdata`, kept between restarts
Health check	`pg_isready`, so scripts can wait for it
Credentials	The same local-only values as `.env.example`

Tests

Kind	How
Unit (most)	As today, with Vitest and the memory store. No database needed.
Environment	`parseEnv()` with crafted inputs: missing, empty, commented out, half pairs, stubs outside the allowlist, partial AWS keys, a temporary key without its session token.
Database URL	`DATABASE_URL` precedence, composition from `DB_*`, special characters in passwords, `schemaFromUrl()`.
Integration	Migrations and the PostgreSQL store against a real database. Skipped when no test database is configured.

The test database is chosen in this order: `TEST_DATABASE_URL` if set; otherwise the main URL with the schema `<schema>_test` if a schema is in use; otherwise the database `<database>_test`. Integration tests refuse to run against anything that is not a test database.

12 Delivery plan and acceptance checks

Four phases, ordered by what each is waiting for. Each phase is its own pull request, and its migration is final once merged.

Phase 0 · Foundation Can start

Typed environment and boot check, database URL module, Kysely and `db:migrate`, local PostgreSQL, reference-data migration, the store, and **rules that survive a restart**.

Phase 1 · People and queues Needs real IDs

The engine carries Genesys user IDs. `person`, `team`, `queue` and `queue_member` are synced from the roster; push subscriptions move into the database.

Phase 2 · Shift records After Phase 1

Shifts, call-outs, incidents, strikes and delivery records, with the write-behind queue and recovery after a restart.

Phase 3 · Raw events After Phase 2

Today's Genesys events with batched inserts, emptied at midnight Eastern. The day's KPIs survive a restart.

Phase 0 is done when

- [ ] `.env.example` copied as it is fails the start-up check, safely and readably.
- [ ] One run of the check reports every failing variable, not only the first.
- [ ] Commented-out optional variables work; empty strings are refused; the pair rules are tested.
- [ ] `DATABASE_URL` precedence, `DB_*` composition and `schemaFromUrl()` are tested.
- [ ] The memory store is refused outside development and test; partial or temporary AWS keys without a session token are refused.
- [ ] The Docker database matches `.env.example`, and `db:make` works with no database settings.
- [ ] `db:migrate` on an empty database creates the Phase 0 tables and reference data; a second run does nothing.
- [ ] An edited threshold is still there after a restart, with its old value kept as an earlier version.
- [ ] `npm run typecheck`, `npm run lint` and `npm test` pass.

13 Frequently asked questions

Why Kysely and not an ORM such as Prisma?

The schema leans on PostgreSQL features (partial unique indexes, partitions, deferrable keys, generated columns) that ORMs hide or do not support. Kysely writes plain SQL with TypeScript checking, and its migrations are ordinary TypeScript files in the repository.

Do I need a database to work on the console?

No. `PERSISTENCE_MODE=memory` runs it exactly as it runs today.

Can I run db:migrate from my laptop against AWS?

Not directly: the database has no public address. Use the deploy script on the EC2 instance, or in an emergency an SSM port-forward (Section 8).

What happens if a migration fails halfway?

It runs in a transaction, so PostgreSQL rolls it back and the database is as it was. The deploy script stops before restarting the app, and the running console is untouched.

Can sharing van-reservation-prod affect Van Reservation?

Only through load, and that is capped: connection limits on the RTM roles, batched writes, and no changes to the instance itself. The RTM roles cannot read or change Van Reservation's data. If the console's load ever matters, it moves to its own instance (Section 4).

Should developers point at the shared instance?

No. It is a production instance. Develop against the local Docker database; only the deployed console connects to `rtm_console`.

Can we run two copies of the console for resilience?

Not yet. Two engines would disagree. A second web tier needs the engine to publish each tick through a broker such as Redis (data architecture document, Section 9).

Will an Admin's rule edits be overwritten by a later migration?

No. Default rules are inserted once, by the migration that creates the table. Changing a default for new installs means a new migration that touches only rules nobody has edited.

Where do the real passwords go?

In AWS Secrets Manager for deployed environments and in `.env.local` on a developer's PC. Neither is ever committed: the repository is public.

14 Quick reference card

FIRST LOCAL RUN

Copy `.env.example` → `.env.local` → `npm run db:up` → `npm run db:migrate` → `npm run dev`.

NEW MIGRATION

`npm run db:make -- add_x` → write `up` and `down` → `db:migrate` → `db:types` → commit both.

DEPLOY (EC2)

`npm ci` → `npm run build` → `npm run db:migrate` as `rtm_migrator` → exit 0 → restart the app.

WHICH URL WINS

`DATABASE_URL` → else `DB_*`. A shell export beats `.env.local`. `DB_SCHEMA` sets `search_path`.

NO DATABASE

`PERSISTENCE_MODE=memory`, development and test only.

NEVER

Edit an applied migration · roll back in production · change the shared instance's settings · turn off TLS verification · commit a secret.

Packages added

Package	Purpose
`kysely`, `pg`	Query builder and PostgreSQL driver (dependencies)
`zod`	Environment validation (dependency)
`kysely-ctl`, `kysely-codegen`, `@types/pg`	Migration commands and type generation (development only)

SETTLED FOR PHASE 2

A shift is one US Eastern day, midnight to midnight, and strikes start again at zero at Eastern midnight. Every strike from 3 on opens its own incident, so strike 5 on the same rule means 3 incidents to investigate. These answers go into the first Phase 2 migration, which cannot be edited once it has run.